Institute Vision and Mission

Vision

To be knowledge hub of providing quality technical education and promoting research for building up of our nation and its contribution for the betterment of humanity

Mission

- To make the best use of state-of-the-art infrastructure to ensure quality technical education
- To develop industrial collaborations to promote innovation and research capabilities
- To inculcate values and ethics to serve humanity

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution
with 'A' Grade
An ISO 21001:2018 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To be a source of knowledge in the field of Computer Science and Engineering to cater to the growing need of industry and society

Mission

- To avail state-of-the art infrastructure for adopting cutting edge technologies and encouraging research activities.
- To promote industrial collaborations for professional competency.
- To nurture social responsibility and ethics to become worthy citizens

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Programme Educational Objectives (PEOs)

Graduates of Computer Science and Engineering Program will

1. Become a globally competent professional in all spheres and pursue higher education world over.
2. Successfully carry forward domain knowledge in computing and allied areas to solve complex real world engineering problems.
3. Continuously upgrade their technical knowledge and expertise to keep pace with the technological revolution.
4. Serve the humanity with social responsibility combined with ethics.

Program Specific Outcomes (PSOs)

Graduates of Computer Science and Engineering Program will be able to:

- Analyze, design and develop computing solutions by applying foundational concepts of Computer Science and Engineering.
- Apply software engineering principles and practices for developing quality software for scientific and business applications.
- To implement cost-effective solutions for the betterment of both industry and society with the technological skills acquired through the Centres of Excellence.

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

22CS601	COMPILER DESIGN (Lab Integrated)	L	T	P	C
		3	0	2	4

OBJECTIVES:

The Course will enable learners to:

- Study the different phases of compiler
- Understand the techniques for tokenization and parsing
- Understand the conversion of source program into an intermediate representation
- Learn the different techniques used for run time environment and code generation
- Analyze various code optimization techniques

UNIT I	INTRODUCTION TO COMPILERS	9+6

Introduction–Structure of a Compiler–Role of the Lexical Analyzer - Input Buffering - Specification of Tokens - Recognition of Tokens–The Lexical Analyzer Generator LEX- Finite Automata - Regular Expressions to NFA-Optimization of DFA based pattern matches - Conversion from NFA to DFA - Minimization of DFA.

List of Exercise/Experiments:

1. Develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, constants, comments, operators etc.). Create a symbol table, while recognizing identifiers.

UNIT II	SYNTAX ANALYSIS	9 + 6

Role of the Parser - Context-free grammars – Derivation Trees – Ambiguity in Grammars and Languages- Writing a grammar – Types of parsing - Top-Down Parsing – Predictive parser or LL(1) Parser –Bottom-Up Parsing – Shift Reduce Parser - LR Parsers - SLR, CLR, LALR Parser - Parser Generators YACC.

List of Exercise/Experiments:

1. Design a lexical analyzer for the given language. The lexical analyzer should ignore redundant spaces, tabs and new lines, comments etc.

UNIT III	INTERMEDIATE CODE GENERATION	9 + 6

Syntax Directed Definitions - Evaluation Orders for Syntax Directed Definitions – Application of Syntax Directed Translation - Intermediate Languages - Syntax Tree – Three Address Code – Implementation of Three address code – Declarations - Translation of Expressions - Type Checking.

List of Exercise/Experiments:

1. Implement a Lexical Analyzer using Lex Tool
2. Design Predictive Parser for the given language

UNIT IV	RUN-TIME ENVIRONMENT AND CODE GENERATION	9 + 6

Run Time Environment: Storage Organization-Storage allocation strategies - Access to nonlocal data on stack – Heap management - Parameter Passing - Issues in the design of Code Generator – Design of simple Code Generator –Register allocation and assignment.

List of Exercise/Experiments:

1. Implement an Arithmetic Calculator using LEX and YACC
2. Generate three address code for a simple program using LEX and YACC.

UNIT V	CODE OPTIMIZATION	9 + 6

Principle sources of optimization – Peep hole Optimization – DAG construction -Basic blocks and flow graph - Optimization in Basic blocks – Data flow analysis.

List of Exercise/Experiments:

1. Generate three address code for a simple program using LEX and YACC.
2. Implement simple code optimization techniques (Constant folding, Strength reduction and Algebraic transformation)
3. Implement back-end of the compiler for which the three address code is given as input and the 8086 assembly language code is produced as output.

TOTAL: 45 +30 = 75 PERIODS

OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Explain the different phases of compiler

CO2: Describe the fundamental components of a compiler.

CO3: Design and implement a lexical analyzer using finite automata and regular expressions.

CO4: Compare various parsing techniques.

CO5: Implement code optimization techniques with simple code generators

CO6: Develop code generation strategies that translate intermediate representations into target machine code.

TEXT BOOK:

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, "Compilers: Principles, Techniques and Tools", Second Edition, Pearson Education Limited, 2014.

REFERENCES:

1. Randy Allen, Ken Kennedy, "Optimizing Compilers for Modern Architectures: A Dependence-based Approach", Morgan Kaufmann Publishers, 2002.
2. Steven S. Muchnick, "Advanced Compiler Design and Implementation", Morgan Kaufmann Publishers - Elsevier Science, India, Indian Reprint, 2003.
3. Keith D Cooper and Linda Torczon, "Engineering a Compiler", Morgan Kaufmann Publishers, Elsevier Science, 2004.
4. V. Raghavan, "Principles of Compiler Design", Tata McGraw Hill Education Publishers, 2010.
5. Allen I. Holub, "Compiler Design in C", Prentice-Hall Software Series, 1993.

LIST OF EQUIPMENTS:

C/C++

COURSE OUTCOMES	
CO1	Explain the different phases of compiler
CO2	Describe the fundamental components of a compiler
CO3	Design and implement a lexical analyzer using finite automata and regular expressions
CO4	Compare various parsing techniques.
CO5	Implement code optimization techniques with simple code generators
CO6	Develop code generation strategies that translate intermediate representations into target machine code.

Course Articulation Matrix

Course Code/ Course Name: 22CS601/ Compiler Design

Course Outcome – Programme Outcome Mapping:

Course Outcomes (COs)	Programme Outcomes (POs)												Programme Specific Outcomes (PSOs)		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	2	-	2	-	-	1	1	2	-	1	3	2	2
CO2	3	2	2	-	2	-	-	1	1	2	-	1	3	2	2
CO3	3	3	3	1	3	-	-	1	2	2	-	2	3	3	3
CO4	3	3	2	1	2	-	-	1	2	2	-	2	3	2	2
CO5	3	3	3	2	3	-	-	2	3	-	-	-	3	3	2
CO6	3	3	3	2	3	-	-	2	2	3	-	2	3	3	2

EXPERIMENTS WITH MAPPED CO

LIST OF EXPERIMENTS

Sl.No.	EXPERIMENT NAME	Mapped CO
1.	Implementation of Symbol Table	CO2
2.	Develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, Constants, Comments, operators etc.)	CO3
3.	Implementation of Lexical Analyzer using LEX Tool	CO3
4.	Design Predictive Parser for the given language	CO4
5.	Generate YACC specification for a few syntactic categories. a) Program to recognize a valid arithmetic expression that uses operator +, - , * and /. b) Program to recognize a valid variable which starts with a letter followed by any number of letters or digits. c) Implementation of Calculator using LEX and YACC	CO4 CO3 CO4
6.	Generate three address code for a simple program using LEX and YACC.	CO5
7.	Implement the back end of the compiler which takes the three address code and produces the 8086 assembly language instructions that can be assembled and run using a 8086 assembler. The target assembly instructions can be simple move, add, sub, jump. Also simple addressing modes are used.	CO6
8.	Implementation of Simple Code Optimization Techniques (Constant Folding., etc.)	CO5

Rubrics for Assessment

	Excellent	**Good**	**Average**	**Poor**
Problem Understanding & Logic Design	Clear understanding with efficient logic (3 marks)	Good logic with minor issues (2 marks)	Partial understanding (1 mark)	unclear understanding (0 mark)
Code Implementation	Error-free, well-structured, meets all requirements (3 marks)	Minor errors, mostly meets objectives (2 marks)	Major errors (1 mark)	incomplete implementation (0 mark)
Output & Viva Explanation	All test cases passed, confident explanation (4 marks)	Some test cases passed, fair explanation (3 marks)	Few cases passed, weak explanation (2 marks)	Attempted, No cases passed, no explanation (1 mark)

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

LIST OF EXPERIMENTS

Sl.NO	NAME OF THE EXPERIMENT
1	Implementation of Symbol Table
2	Develop a lexical analyzer to recognize a few patterns in C. (Ex. identifiers, Constants, Comments, operators etc.)
3	Implementation of Lexical Analyzer using LEX Tool
4	Design Predictive Parser for the given language
5	Generate YACC specification for a few syntactic categories. a) Program to recognize a valid arithmetic expression that uses operator +, - , * and /. b) Program to recognize a valid variable which starts with a letter followed by any number of letters or digits. c) Implementation of Calculator using LEX and YACC
6	Generate three address code for a simple program using LEX and YACC.
7	Implement the back end of the compiler which takes the three address code and produces the 8086 assembly language instructions
8	Implementation of Simple Code Optimization Techniques (Constant Folding., etc.)

Ex. No: 01 Date :	IMPLEMENTATION OF SYMBOL TABLE

AIM:

To write a C program to implement Symbol Table with functions to create, insert, modify, search, and display.

ALGORITHM:

STEP 1: Start the program for performing insert, display, delete, search and modify option in symbol table

STEP 2: Define the structure of the Symbol Table

STEP 3: Enter the choice for performing the operations in the symbol Table

STEP 4: If the entered choice is 1, search the symbol table for the symbol to be inserted. If the symbol is already present, it displays "Duplicate Symbol". Else, insert the symbol and the corresponding address in the symbol table.

STEP 5: If the entered choice is 2, the symbols present in the symbol table are displayed.

STEP 6: If the entered choice is 3, the symbol is searched in the symbol table. If it is not found in the symbol table it displays "Label Not found".

STEP 7: If the entered choice is 4, the symbol to be modified is searched in the symbol table. The address of the label can be modified.

STEP 8: Enter choice 5 to exit the program.

PROGRAM:

```
# include <stdio.h>
# include <conio.h>
# include <alloc.h>
# include <string.h>
# define null 0
     int size=0;
void insert();
int search(char *);
void modify();
void display();
struct symbtab
{
   char label[10];
   int addr;
   struct symbtab *next;
};
struct symbtab *first,*last;
void main()
{
   int op;
   int y;
   char la[10];
   clrscr();
   do       {
         printf (" \nSYMBOL TABLE IMPLEMENTATION\n ");
         printf ("1. INSERT\n ");
```

```
        printf ("2. DISPLAY\n ");
        printf ("3. SEARCH\n ");
        printf ("4. MODIFY\n ");
        printf ("5. END\n");
        printf ("Enter your option : ");
        scanf ("%d", &op);

switch(op)        {
case 1:
          insert();
          display();
          break;
case 2:
        display();
        break;
case 3:
            printf("Enter the label to be searched : ");
            scanf("%s", la);
            y=search(la);
            if(y==1)
                    printf( "The label is already in the symbol Table");
            else
                    printf( "The label is not found in the symbol table");
            break;
   case 4:
            modify();
            display();
            break;
   case 5:
            break;
   }
}
while(op<5);
getch();
}
void insert()       {
            int n;
            char l[10];
            printf("Enter the label : ");
            scanf("%s",l);
            n=search(l);
            if(n==1)  {
                    printf("The label already exists. Duplicate cant be inserted\n");
            }
            else {
                    struct symbtab *p;
                    p=(struct symbtab*) malloc( sizeof( struct symbtab));
                    strcpy( p -> label,l);
                    printf( "Enter the address : ");
                    scanf( "%d",  &p -> addr);
```

```
                p -> next=null;
                if( size==0)     {
                        first=p;
                        last=p;
                }
                else     {
                        last->next=p;
                        last=p;
                }
                size++;
            }
 }

 void display()                {
        int i;
        struct symbtab *p;
        p=first;
        printf("LABEL\t ADDRESS \n");
        for(i=0; i<size; i++)
            {
            printf( "%s \t %d \n",  p -> label,  p -> addr);
            p = p -> next;
            }
}

int search (char lab[])      {
    int i, flag=0;
    struct symbtab *p;
    p = first;
    for(i=0; i<size; i++)    {
        if (strcmp ( p->l abel, lab) ==0)
            flag=1;
        p=p->next;
    }
        return flag;
    }

 void modify()     {
    char l[10],  nl[10];
    int add, choice,   i,  s;
    struct symbtab *p;
    p = first;
    printf( "Enter the label whose address is to modified\n");
    scanf( "%s", l);
    printf("Enter the new address\n");
    scanf("%d", &add);
    s = search(l);
    if (s==0)         {
        printf("No such label");
        }
```

```
    else

    {
        for(i=0;i<size;i++) {
            if(strcmp ( p -> label, l)==0)
                p -> addr =add;
        p= p -> next;
        }
    }
}
```

SAMPLE INPUT & OUTPUT:

```
SYMBOL TABLE IMPLEMENTATION
1. INSERT
2. DISPLAY
3. SEARCH
4. MODIFY
5. END
Enter your option: 1
Enter the label: ONE
Enter the type : INT
Enter the address: 1000
LABEL  TYPE     ADDRESS
ONE      INT          1000


SYMBOL TABLE IMPLEMENTATION
1. INSERT
2. DISPLAY
3. SEARCH
4. MODIFY
5. END
Enter your option: 1
Enter the label: TWO
Enter the type : INT
Enter the address: 2000
LABEL   TYPE      ADDRESS
ONE       INT        1000
TWO       INT        2000



SYMBOL TABLE IMPLEMENTATION
1. INSERT
2. DISPLAY
3. SEARCH
4. MODIFY
5. END
```

```
Enter your option: 2
LABEL   TYPE      ADDRESS
ONE       INT       1000
TWO       INT       2000
THREE    FLOAT    4000

SYMBOL TABLE IMPLEMENTATION
1. INSERT
2. DISPLAY
3. SEARCH
4. MODIFY
5. END
Enter your option: 4
Enter the label whose address is to modified
THREE
Enter the new address     : 3000
LABEL   TYPE      ADDRESS
ONE       INT       1000
TWO       INT       2000
THREE    FLOAT    3000
```

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

Thus the program is executed and output is verified for implementing Symbol Table.

Ex. No: 02 Date :	DEVELOP A LEXICAL ANALYZER TO RECOGNIZE A FEW PATTERNS IN C

AIM:

To Write a program for implementing Lexical Analyzer in 'C'.

ALGORITHM:

STEP 1: Create files for storing keywords, Operators and a program as an input.
STEP2: Read the source file as input.
STEP3: Scan the first character
STEP5: Check the character begin with alphabet or digit using the functions isalpha() and isdigit().
STEP6: If isdigit() returns true then print it as a constant.
STEP7: If isalpha() returns true check it with keyword file content . If anyone the keyword matches then print it as keyword otherwise print it as an identifier.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<ctype.h>
int main()
{
        FILE *fsur, *fkey, *fopr;
        char c,str[20],ky[20],op[20],temp[20];
        int i, set=0;
        clrscr();
        fsur=fopen("source.txt","r");
        fkey=fopen("keyword.txt","r");
        fopr=fopen("operator.txt","r");
        while(feof(fsur)==0)
        {
           c=fgetc(fsur);
        //Keywords and Identifiers
           if(isalpha(c))
           {
                for(i=0;(isalpha(c))||(isdigit(c))||(c=='_');i++)
                {
                        str[i]=c;
                        c=fgetc(fsur);
                }
                str[i]='\0';
                rewind(fkey);
                while(feof(fkey)==0)
                {
                        fscanf(fkey,"%s",ky);
                        if(strcmp(ky,str)==0)                    {
                                printf("\n%s is a keyword",str);
                                set=1;
                                break;
                        }
```

```
            }
            if(set==0)
                    printf("\n%s is an identifier",str);
            set=0;
        }
    //Numeric literals
        if(isdigit(c))          {
            for(i=0;(isdigit(c))||(c=='.');i++)
            {
                    str[i]=c;
                    c=fgetc(fsur);
            }
            str[i]='\0';
            printf("\n%s is a numeric literal",str);
        }
    //String literals
        if(c=='\"')
        {
            c=fgetc(fsur);
            for(i=0;c!='\";i++)
            {
                    str[i]=c;
                    c=fgetc(fsur);
            }
            str[i]='\0';
            printf("\n\"%s\" is a string literal",str);
        }
    //Character literal
        if(c=='\')        {
            c=fgetc(fsur);
            if(c=='\\')
            {
               c=fgetc(fsur);
               printf("\n\\%c is a character literal",c);
            }
            else
               printf("\n%c is a character literal",c);
            if((c=fgetc(fsur))!='\')
               printf("\nInvalid character literal");
        }
    //Preprocessor
        if(c=='#')
        {
            printf("\n%c is a preprocessor symbol",c);
        }
    //Brackets
        if(c=='('||c=='{')
            printf("\n%c is an opening braces",c);
        if(c==')'||c=='}')
            printf("\n%c is a closing braces",c);
    //Punctuation symbols
        if(c==',')
            printf("\n%c is a puntuation",c);
        if(c==';')
            printf("\n%c is end of statement",c);
```

```
        //comment lines
          if(c=='/')        {
                printf("\ncomment lines\n");
                c=fgetc(fsur);
                if(c=='/')
                {
                  while(c!='\n')   {
                        c=fgetc(fsur);
                        printf("%c",c);
                  }
                }
                else if(c=='*')
                {
                  for(;;)
                  {
                        c=fgetc(fsur);
                        if(c=='*')
                        {
                          c=fgetc(fsur);
                          if(c=='/') break;
                        }
                        printf("%c",c);
                  }
                  c=getc(fsur);                    }            }
         //operators
          rewind(fopr);
          temp[0]=c;
          temp[1]='\0';
          while(feof(fopr)==0)            {
                fscanf(fopr,"%s",op);
                if(strcmp(op,temp)==0)                  {
                  c=fgetc(fsur);
                  temp[1]=c;
                  temp[2]='\0';
                  rewind(fopr);
                  while(feof(fopr)==0)                  {
                        fscanf(fopr,"%s",op);
                        if(strcmp(op,temp)==0)                          {
                          printf("\n%s is an operator",temp);
                          set=1;
                          break;        }                  }
                  if(set==0)
                  {
                        printf("\n%c is an operator",temp[0]);
                        ungetc(c,fsur);
                        break;                }                }            }
          set=0;        }
        fcloseall();
        getch();
        return 0;
}
```

SAMPLE INPUT & OUTPUT

Input: SOURCE.TXT

```
 // demo program
# define N 30
int main()        {
  float  a  = 61.37;
  char  str1 = 's';
  printf("%c", str1);
 return 0;         }
```

KEYWORD.TXT

```
define
int
main
float
char
printf
return
if
while
for
```

OPERATOR.TXT

```
+
-
*
/
=
%
==
>
<
>=
<=
```

OUTPUT:

Comment lines
demo program
is a preprocessor symbol
define is a keyword
N is an identifier
20 is a numeric literal
int is a keyword
main is a keyword
(is an opening braces
) is an closing braces
{ is an opening braces
float is a keyword
a is an identifier
= is an operator
61.37 is a numeric literal

; is end of statement
char is a keyword
str1 is an identifier
= is an operator
s is a character literal
printf is a keyword
(is an opening braces
“%c” is a string literal
, is a punctuation
strl is an identifier
) is an closing braces
; is end of statement
return is a keyword

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT:
Thus the program is executed and output is verified for implementing Lexical **Analyzer** in ‘C’.

Ex. No: 03 Date :	IMPLEMENTATION OF LEXICAL ANALYZER USING LEX TOOL

AIM:

To Write a Program to implement the Lexical Analyzer using LEX Tool.

ALGORITHM:

STEP 1: Define and include necessary declarations and Regular definitions
STEP 2: Define the Translation Rule section for Keywords, Identifiers, Constants, Operators and Literal Strings
STEP 3: Read the input from a file.
STEP 4: Lex tool will compare the input with patterns defined under Translation rule section
STEP 5: If any of the pattern matches with the input respective action will be performed.
STEP 6: End.

PROGRAM:

```
//Program to implement Lexical Analyzer using LEX Tool
%{
#include<math.h>
%}
%{
#include<math.h>
%}
digit [0-9]
id [A-Za-z][A-Za-z0-9]*
li["']
%%
void|main|int|printf     {printf("\nA keyword:%s\n",yytext);}

{digit}+                 { printf("\nAn integer:%s(%d)\n",yytext,atoi(yytext));}

{digit}+"."{digit}*      {printf("\nA float: %s (%g)\n",yytext,atof(yytext));}

{id}                     {printf("\nAn identifier:%s\n",yytext)};

"+"|"-"|"*"|"/"|"="       printf("\nAn operator:%s\n",yytext);

";"|"("|")"|":"|","|"}"|"{" {printf("\n%s is a punctuation\n",yytext);}

#{id}"<"{id}"."{id}">" {printf("\n%s is a preprocessor\n",yytext);}

{li}"%"{id}+{li}         {printf("\n%s is a literal string\n",yytext);}

{li}{id}+{li}            {printf("\n%s is a literal string\n",yytext);}
%%
main(argc,argv)
int argc;
char **argv; {
        yyin=fopen("sha.c","r");
```

```
        yylex();
}
```

How to execute:

```
$lex ss.l   - will create a file called lex.yy.c
$cc lex.yy.c –lfl
$./a.out
```

SAMPLE INPUT & OUTPUT

Input:

Sha.c

```
# include<stdio.h>
Void main()
{
    Int a;
    A= 10 +2;
    printf("%d", a);
}
```

Output:

```
# include<stdio.h>  is a preprocessor
A keyword: void
A keyword: main
(   is a punctuation
)   is a punctuation
{    is a punctuation
A keyword: int
An identifier:  a
;  is a punctuation
An identifier :  a
An operator :  =
An integer:  10
An operator : +
An integer:  2
;  is a punctuation
A keyword : printf
(    is a punctuation
"%d"   is a literal string  etc.,
```

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

Thus the program is executed and output is verified to implement the Lexical Analyzer using LEX Tool.

Ex. No: 04 Date :	DESIGN A PREDICTIVE PARSER FOR THE GIVEN LANGUAGE

AIM

To design a Predictive Parser for the given Language

ALGORITHM

Input: ParsingTable, InputString

1. Initialize:
   - Input pointer to the start of InputString.
   - Initialize the stack with starting symbol 'S' and end marker '$'.

2. While the stack is not empty:
   - Peek at the top of the stack (stackTop).
   - If stackTop is a terminal:
     - If stackTop matches the current input token:
       - Pop stackTop from the stack.
       - Advance the input pointer.
     - Else, report an error and halt.
   - Else if stackTop is a non-terminal:
     - Look up the parsing table entry ParsingTable[stackTop][currentInputToken].
     - If an entry is found:
       - Pop the non-terminal off the stack.
       - Push the RHS of the corresponding production rule in reverse order onto the stack.
     - Else, report a syntax error and halt.

End

PROGRAM:

```
#include <stdio.h>

#include <string.h>

char prol[7][10] = { "S", "A", "A", "B", "B", "C", "C" };
char pror[7][10] = { "A", "Bb", "Cd", "aB", "@", "Cc", "@" };
char prod[7][10] = { "S->A", "A->Bb", "A->Cd", "B->aB", "B->@", "C->Cc", "C-
>@" };
char first[7][10] = { "abcd", "ab", "cd", "a@", "@", "c@", "@" };
char follow[7][10] = { "$", "$", "$", "a$", "b$", "c$", "d$" }; char table[5][6][10];
int numr(char c)   {
  switch (c){
      case 'S': return 0;
```

```
      case 'A': return 1;

      case 'B': return 2;

      case 'C': return 3;

      case 'a': return 0;

      case 'b': return 1;

      case 'c': return 2;

      case 'd': return 3;

      case '$': return 4;
  }
  return (2);
}
int main()
{
  int i, j, k;

    for (i = 0; i < 5; i++) for (j = 0; j < 6; j++)
      strcpy(table[i][j], " ");

  printf("The following grammar is used for Parsing Table:\n"); for (i = 0; i < 7; i++)
    printf("%s\n", prod[i]);

  printf("\nPredictive parsing table:\n"); fflush(stdin);

  for (i = 0; i < 7; i++)
  {
    k = strlen(first[i]); for (j = 0; j < 10; j++)
      if (first[i][j] != '@')
        strcpy(table[numr(prol[i][0]) + 1][numr(first[i][j]) + 1], prod[i]);
  }

  for (i = 0; i < 7; i++)
  {
    if (strlen(pror[i]) == 1)
    {
      if (pror[i][0] == '@')
      {
        k = strlen(follow[i]); for (j = 0; j < k; j++)
          strcpy(table[numr(prol[i][0]) + 1][numr(follow[i][j]) + 1], prod[i]);
      }
    }
  }

  strcpy(table[0][0], " ");

  strcpy(table[0][1], "a");

  strcpy(table[0][2], "b");

  strcpy(table[0][3], "c");
```

```
    strcpy(table[0][4], "d");

    strcpy(table[0][5], "$");

    strcpy(table[1][0], "S");

    strcpy(table[2][0], "A");

    strcpy(table[3][0], "B");

    strcpy(table[4][0], "C");

    printf("\n---------------------------------- \n");

      for (i = 0; i < 5; i++) for (j = 0; j < 6; j++)
      {
        printf("%-10s", table[i][j]); if (j == 5)
          printf("\n------------------------------------------------------- \n");
      }
  }
```

Sample Output :

```
The following grammar is used for Parsing Table: S->A
A->Bb A->Cd B->aB B->@ C->Cc C->@

Predictive parsing table:

-------------------------------------
        a    b      c      d      $
-------------------------------------
        S      S->A   S->A   S->A     S->A
-------------------------------------
          A    A->Bb   A->Bb   A->CdA->Cd
-------------------------------------
       B        B->aB   B->@          B->@     B->@
-------------------------------------
        C          C->@   C->@   C->@

 -------------------------------------
        -
```

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

This a predictive Parser for the C programming Language was successfully executed.

Ex. No: 05 a. Date :	GENERATE YACC SPECIFICATION FOR A FEW SYNTACTIC CATEGORIES. a) PROGRAM TO RECOGNIZE A VALID ARITHMETIC EXPRESSION THAT USES OPERATOR +, - , * AND /

AIM :

To Write a program to implement a calculator using YACC Tool.

ALGORITHM:

STEP 1: Define and include necessary 'C' declarations and token definitions
STEP 2: Define the translation rules and the input structure specifications for the grammar rules
STEP 3: Define the functions that are invoked in the rules
STEP 4: Read the input
STEP 5: If the given input matches with the defined rules , the YACC tool executes the respective actions.
STEP 6: End

PROGRAM:

```
//Program to implement calculator using yacc Tool

%{
#define YYSTYPE double
%}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
list: /*nothing*/
     |list '\n'
     |list expr '\n' {printf("\t%.8g\n",$2);}
     ;
expr: NUMBER {$$=$1;}
       |expr '+' expr {$$=$1+$3;}
       |expr '-' expr {$$=$1-$3;}
       |expr '*' expr {$$=$1*$3;}
       |expr '/' expr {$$=$1/$3;}
       |'(' expr ')'  {$$=$2;}
       ;
%%
#include<stdio.h>
#include<ctype.h>
char *progname;
int lineno=1;
main(argc,argv)
 char *argv[];
{
           progname=argv[0];
           yyparse();
          }
```

```
yylex()
{
        int c;
        while((c=getchar())==' '||c=='\t')
          ;
        if(c==EOF)
         return 0;
        if(c=='.'||isdigit(c))
        {
                ungetc(c,stdin);
                scanf("%lf",&yylval);
                return NUMBER;
        }
        if(c=='\n')
                lineno++;
        return(c);
}
```

How to execute

$yacc ss.y (create a file called y.tab.c)
$cc y.tab.c –ly
$./a.out

SAMPLE INPUT & OUTPUT

Input:
5 + 6

Output:
11

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Result
Thus the program is executed and output is verified to implement the Lexical Analyzer using LEX Tool.

Ex. No: 05 b. Date :	GENERATE YACC SPECIFICATION FOR A FEW SYNTACTIC CATEGORIES. b) PROGRAM TO RECOGNIZE A VALID VARIABLE WHICH STARTS WITH A LETTER FOLLOWED BY ANY NUMBER OF LETTERS OR DIGITS.

AIM :

To Write a program to recognize a valid variable which starts with a letter followed by any number of letters or digits.

ALGORITHM:

STEP 1: Write one lex file and one YACC file
STEP 2: Define the tokens in lex file and the translation rules and the input structure specifications for the grammar rules in YACC files
STEP 3: Define the functions that are invoked in the rules
STEP 4: Read the input by using the function yyparse
STEP 5: If the given input matches with the defined rules (A specifies that LETTER token is followed by B, B specifies that LETTER token is followed by any number of LETTERS or DIGITS) , the YACC tool specifies it as a valid variable.
STEP 6: End

PROGRAM:

4b.l file

```
%{      #include "y.tab.h"      %}
%%
[0-9]+ {return DIGIT;}
[a-zA-Z]+ {return LETTER;}
[ \t] {;}
\n { return 0;}
. {return yytext[0];}
%%
```

4b.y file

```
%{      #include<stdio.h>      %}
%token DIGIT LETTER
%%
stmt:A
    ;
A: LETTER B
 ;
B: LETTER B
 | DIGIT B
 | LETTER
 | DIGIT
 ;
%%
void main()     {
                printf("enter string \n");
                yyparse();
                printf("valid");
                exit(0);
        }
```

```
void yyerror()          {
printf("invalid");
exit(0);
}
```

How to execute:

$yacc -d prog.y - will create a file called y.tab.c ,y.tab.h

$cc y.tab.c –1ll

$./a.out

SAMPLE OUTPUT:

Input : Enter a variable : **asd12**

Output : Accepted

Input : Enter a variable : **12adr**

Output : Rejected

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

Thus above program executed and output is verified to to recognize a valid variable which starts with a letter followed by any number of letters or digits.

Ex. No: 05 c. Date :	GENERATE YACC SPECIFICATION FOR A FEW SYNTACTIC CATEGORIES. C) IMPLEMENTATION OF CALCULATOR USING LEX AND YACC

AIM:

To write a program to implement scientific calculator using LEX and YACC.

ALGORITHM:

For Lex Program sample.l

STEP 1: Include the necessary library files for token definitions and 'C' declarations

STEP 2: Define the rules for regular definitions and patterns

STEP 3: Define the functions invoked in the translation rules

For Yacc Program sample.y

STEP 1: Include the necessary declarations for yacc specifications and 'C' headers

STEP 2: Define the translation rules and the input structure specifications for the grammar rules

STEP 3: Define the functions that are invoked in the rules

STEP 4: Read the input

STEP 5: If the given input matches with the defined rules, the yacc tool executes the respective actions.

STEP 6: End.

PROGRAM:

```
//Program to implement scientific calculator using LEX and YACC
//lexcalc.l
%{
        #include "y.tab.h"
        #include <math.h>
%}
%%
([0-9]+|([0-9]*\.[0-9]+)([eE][-+]?[0-9]+)?) { yylval.dval = atof(yytext); return
NUMBER; }
log |
LOG     { return LOG; }
ln      { return nLOG; }
sin |
SINE    { return SINE; }
cos |
COS     { return COS; }
tan |
TAN     { return TAN; }
mem     { return MEM; }
[\t];
\$      { return 0; }
\n|. return yytext[0];
%%
//yaccalc.y
```

```
%{
        double memvar;
        %}
        %union  {  double dval;
}
        %token <dval> NUMBER
        %token <dval> MEM
        %token LOG SINE nLOG COS TAN
        %left '-' '+'
        %left '*' '/'
        %right '^'
        %left LOG SINE nLOG COS TAN
        %nonassoc UMINUS
        %type <dval>expr
        %%
        start : statement '\n'
              | start statement '\n'
              ;
        statement : MEM '=' expr { memvar = $3; }
              | expr { printf("Answer = %g\n",$1); }
              ;
        expr:     expr '+' expr { $$ = $1 + $3; }
              | expr '-' expr { $$ = $1 - $3; }
              | expr '*' expr { $$ = $1 * $3; }
              | expr '/' expr
              {
                    if($3==0)
                          printf("Divide by zero");
                    else
                          $$ = $1 / $3;
              }
              | expr '^' expr { $$ = pow($1,$3); }
              ;
        expr : '-' expr %prec UMINUS { $$ = -$2; }
              | '(' expr ')' { $$ = $2; }
              | LOG expr { $$ = log($2)/log(10); }
              | nLOG expr { $$ = log($2); }
              | SINE expr { $$ = sin($2 * 3.1415/180); }
              | COS expr { $$ = cos($2*3.1415/180); }
              | TAN expr { $$ = tan($2*3.1415/180); }
              | NUMBER  { $$ = $1; }
              | MEM { $$ = memvar; }
              ;
        %%
        main()  {
              printf("Enter the Expression\n");
              yyparse();
}
```

SAMPLE INPUT & OUTPUT

Input:

Enter the Expression
log100

Output:

Answer = 2

Input:

Enter the Expression
tan45

Output:

Answer = 0.999954

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

Thus the program is executed and output is verified for implement scientific calculator using LEX and YACC.

Ex. No: 06 Date :	**GENERATE THREE ADDRESS CODE FOR A SIMPLE PROGRAM USING LEX AND YACC**

AIM

To generate three address code for a simple program.

ALGORITHM

Input: An arithmetic expression or program statement.

Initialize: Create a counter for temporary variable names (e.g., t1, t2, etc.).

Parse Expression: Parse the input into an abstract syntax tree (AST) or expression tree.

Traverse the Tree:

- For each binary operator node:
  - Create a temporary variable.
  - Generate the TAC instruction to compute the result.
- For each unary operator node (e.g., negation):
  - Generate the appropriate TAC instruction with one operand.
- For variable assignments, directly generate the assignment to a temporary variable.

Output: Print or return the generated Three-Address Code.

PROGRAM:

Lex File (lex.l)

```
%{
#include "y.tab.h"
%}
%%
[0-9]+              { yylval = atoi(yytext); return NUMBER; }
[a-zA-Z][a-zA-Z0-9]*   { yylval = strdup(yytext); return ID; }
"+"                 { return PLUS; }
"-"                 { return MINUS; }
"*"                 { return MUL; }
"/"                 { return DIV; }
"="                 { return ASSIGN; }
"("                 { return LPAREN; }
")"                 { return RPAREN; }
[ \t\n]             { /* skip whitespace */ }
.                   { return yytext[0]; }
%%

int yywrap() { return 1; }
```

Yacc File (yacc.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int temp_count = 1;
void generate_code(char *op, char *arg1, char *arg2, char *result) {
    if (result != NULL) {
        printf("%s = %s %s %s\n", result, arg1, op, arg2);
    } else {
        printf("%s = %s %s %s\n", arg1, arg1, op, arg2);
    }
}
%}
%union {
    int num;
    char* id;
}
%token <num> NUMBER
%token <id> ID
%token PLUS MINUS MUL DIV ASSIGN LPAREN RPAREN
%type <num> expression term factor

%%
program:
    program statement
    | statement
    ;

statement:
    ID ASSIGN expression {
        generate_code("=", $3, "", $1);
    }
    ;

expression:
    expression PLUS term {
        char temp[10];
        sprintf(temp, "t%d", temp_count++);
        generate_code("+", $1, $3, temp);
        $$ = strdup(temp);
    }
    | expression MINUS term {
        char temp[10];
        sprintf(temp, "t%d", temp_count++);
        generate_code("-", $1, $3, temp);
        $$ = strdup(temp);
    }
    | term { $$ = $1; }
    ;

term:
    term MUL factor {
        char temp[10];
```

```
            sprintf(temp, "t%d", temp_count++);
            generate_code("*", $1, $3, temp);
            $$ = strdup(temp);
        }
        | term DIV factor {
            char temp[10];
            sprintf(temp, "t%d", temp_count++);
            generate_code("/", $1, $3, temp);
            $$ = strdup(temp);
        }
        | factor { $$ = $1; }
        ;
    factor:
        NUMBER {
            char temp[10];
            sprintf(temp, "t%d", temp_count++);
            generate_code("", $1, "", temp);
            $$ = strdup(temp);
        }
        | ID { $$ = $1; }
        | LPAREN expression RPAREN { $$ = $2; }
        ;
    %%
    int main() {
        yyparse();
        return 0;
    }
```

Generate Lex and Yacc Files:

Run the following commands to generate the necessary files.

```
lex lex.l          # Generates lex.yy.c
yacc -d yacc.y     # Generates y.tab.c and y.tab.h
gcc lex.yy.c y.tab.c -o parser -ll -ly  # Compile the Lex and Yacc output
```

Run the Program: Execute the parser with an input file containing a simple program.

a = b + c * d

Run the parser:

```
./parser < input.txt
```

Output

For the input:

a = b + c * d

The generated Three-Address Code will be:

t1 = c * d

t2 = b + t1

a = t2

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT

The three address code for the given program was generated .

Ex. No: 07 Date :	**IMPLEMENT THE BACK END OF THE COMPILER WHICH TAKES THE THREE ADDRESS CODE AND PRODUCES THE 8086 ASSEMBLY LANGUAGE INSTRUCTIONS THAT CAN BE ASSEMBLED AND RUN USING A 8086 ASSEMBLER. THE TARGET ASSEMBLY INSTRUCTIONS CAN BE SIMPLE MOVE, ADD, SUB, JUMP. ALSO SIMPLE ADDRESSING MODES ARE USED.**

AIM:

To Write a Program to implement the Back End of a Compiler.

ALGORITHM:

STEP 1: Read the intermediate codes as input from the user.
STEP 2: Read and check each and every intermediate code in order to find the operator involved in it.
STEP 3: Based on the operator, we can print the mnemonics names for assembly code instruction. i.e, if the operator involved is '+' then we can print the equivalent mnemonic name as ADD.
STEP 4: Similarly do steps 2 & 3 for all the intermediate codes given.
STEP 5: END.

PROGRAM:

```
//Program To Implement Back End of A Compiler
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int n,rlen=0;
char icode[25][25],reg[25][25];
int isopr(char c)
{
   if(c=='+'||c=='-'||c=='*'||c=='/'||c=='='||c=='\0')
      return 1;
   return 0;
}
int getregno(char *tmp)
{
   int i;
   for(i=0;i<rlen;i++)
   {
      if(strcmp(tmp,reg[i]) == 0)
         return i;
   }
   return -1;
}
int addtoreg(char *tmp)
{
   strcpy(reg[rlen],tmp);
   rlen++;
   return (rlen-1);
}
```

```
void genopr(char op)
{
    switch(op)
    {
        case '+':
            printf("\nADD ");
            break;
        case '-':
            printf("\nSUB ");
            break;
        case '*':
            printf("\nMUL ");
            break;
        case '/':
            printf("\nDIV ");
            break;
    }
}
int main()
{
    int i,j,k,rno1,rno2;
    char tmp[25];
    printf("Enter the number of Intermediate code Entries : ");
    scanf("%d",&n);
    printf("\nEnter the Intermediate code\n");
    for(i=0;i<n;i++)
        scanf("%s",icode[i]);
    for(i=0;i<n-1;i++)
    {
        for(j=0;icode[i][j]!='=';j++);
        if(icode[i][j+1]=='-')
        {
            strcpy(tmp,"00");
            j++;
        }
        else
        {
            for(k=0,j=j+1;isopr(icode[i][j])==0;k++,j++)
            tmp[k]=icode[i][j];
            tmp[k]='\0';
        }
        rno1=getregno(tmp);
        if(rno1==-1)
        {
            rno1=addtoreg(tmp);
            if(isdigit(tmp[0]))
                printf("\nMOV #%s,R%d",tmp,rno1);
            else
                printf("\nMOV %s,R%d",tmp,rno1);
        }
        else if(tmp[0]=='t')
        {
            rno2=addtoreg(tmp);
            printf("\nMOV R%d,R%d",rno1,rno2);
        }
```

```
            genopr(icode[i][j]);
            for(k=0,j=j+1;isopr(icode[i][j])==0;k++,j++)
                tmp[k]=icode[i][j];
            tmp[k]='\0';
            rno2=getregno(tmp);
            if(rno2==-1)
            {
                if(isdigit(tmp[0]))
                    printf("#%s,R%d",tmp,rno1);
                else
                    printf("%s,R%d",tmp,rno1);
            }
            else
                printf("R%d,R%d",rno2,rno1);
            for(k=0;isopr(icode[i][k])==0;k++)
                tmp[k]=icode[i][k];
            tmp[k]='\0';
            strcpy(reg[rno1],tmp);
        }
        for(k=0;isopr(icode[i][k])==0;k++)
            tmp[k]=icode[i][k];
        tmp[k]='\0';
        printf("\nMOV R%d,%s\n",rno1,tmp);
        return 0;
    }
```

SAMPLE INPUT & OUTPUT

Input:

```
Enter the number of Intermediate Code Entries   :   3
Enter the Intermediate Code
t1  =  a + b
t2  =  25 * t1
ans = t2
```

Output:

```
MOV  a, R0
ADD   b, R0
MOV   #25, R1
MUL   R0,R1
MOV   R1, ans
```

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT:

Thus the program is executed and output is verified for implement Backend of a compiler.

Ex. No: 08 Date :	IMPLEMENTATION OF SIMPLE CODE OPTIMIZATION TECHNIQUES (CONSTANT FOLDING., ETC.)

AIM:

To write a program for implementation of Simple Code Optimization Techniques (Constant Folding., etc.)

ALGORITHM:

STEP 1: Generate the program for factorial program using for and do-while loop to specify optimization technique and with constant variable j.

STEP 2: In for loop variable initialization is activated first and the condition is checked next. If the condition is true the corresponding statements are executed and specified increment / decrement operation is performed.

STEP 3: The 'for' loop operation is activated till the condition failure.

STEP 4: In do-while loop the variable is initialized and the statements are executed then the condition checking and increment / decrement operation is performed.

STEP 5: When comparing both for and do-while loop for optimization do while is best because first the statement execution is done then only the condition is checked. So, during the statement execution itself we can find the inconvenience of the result and no need to wait for the specified condition result. When compared with variable constant assigning can be done outside the Loop

STEP 6: Finally when considering Code Optimization in loop do-while is best with respect to performance.

PROGRAM:

```
#include<stdio.h>
#include<string.h>
#include<strings.h>
//#include<conio.h>
#include<stdlib.h>
#include<ctype.h>
struct ConsFold
{
        char new_str[10];
        char str[10];
}opt_Data[20];
void ReadInput(char  Buffer[],FILE *Out_file);
int Gen_token(char str[],char Tokens[][10]);
int New_Index=0;

int main()
 {
        FILE *In_file,*Out_file;
        char Buffer[100],ch;
        int i=0;
        In_file = fopen("code.txt","r");
        Out_file = fopen("output.txt","w");
        while(1)
```

```
        {
                ch= fgetc(In_file);
                i=0;
                while(1)
                {
                        if(ch=='\n')
                        break;
                        Buffer[i++]=ch;
                        ch = fgetc(In_file);
                        if(ch==EOF)
                        break;
                }
                if(ch==EOF)
                break;
                Buffer[i]='\0';
                ReadInput(Buffer,Out_file);
        }
    return 0;
}
void ReadInput(char Buffer[],FILE *Out_file)
{
        char temp[100],Token[10][10];
        int n,i,j,flag=0;
        strcpy(temp,Buffer);
        n= Gen_token(temp,Token);
        for(i=0;i<n;i++)
        {
                if(!strcmp(Token[i],"="))
                {
                        if(isdigit(Token[i+1][0])||Token[i+1][0]=='.')
                        {
                        flag=1;
                        strcpy(opt_Data[New_Index].new_str,Token[i-1]);
                        strcpy(opt_Data[New_Index++].str,Token[i+1]);
                        }
                }
        }
        if(!flag)
        {
                for(i=0;i<New_Index;i++)
                {
                        for(j=0;j<n;j++)
                        {
                                if(!strcmp(opt_Data[i].new_str,Token[j]))
                                strcpy(Token[j],opt_Data[i].str);
                        }
                }
        }
        fflush(Out_file);
        strcpy(temp,"");
        for(i=0;i<n;i++)
        {
                strcat(temp,Token[i]);
                if(Token[i+1][0] !=','|| Token[i+1][0] != ',')
                strcat(temp,"\n\0");
```

```
                        fwrite(&temp,flag,strlen(temp),Out_file);
                }
        }
        int Gen_token(char str[],char Token[][10])
        {
                int i=0,j=0,k=0;
                while(str[k] != '\0')
                {
                j=0;
                while(str[k] ==' '|| str[k] == '\t')
                k++;
                while(str[k] != ' '&& str[k] != '\0' && str[k] !='='&& str[k] != '/'&&
                str[k] !='+'&& str[k] != '-'&& str[k] !='*'&& str[k] != ','&& str[k] !=
                ';')
                Token[i][j++] = str[k++];
                Token[i++][j]= '\0';
                if(str[k] == '=' || str[k] == '/' || str[k] == '+' || str[k] == '=' || str[k] =='*' ||
                str[k] == ',' || str[k] == ';')
                {
                        Token[i][0] = str[k++];
                        Token[i++][1] ='\0';
                }
                if(str[k] == '\0')
                break;
                }
                return i;
        }
```

SAMPLE OUTPUT

Before Code optimization: code.txt

```
#include int main()
{ int i, n,j;
 int fact=1;
cout<<"\n Enter a number: ";
cin>>n;
for(i=n ; i>=1 ; i--)
j=23+3;
fact = fact * i;
cout<<"The factoral value is: ";
cout<<j;
```

After code Optimization: output.txt

```
#include void main()
{ clrscr();
int n,f,j=26;
 f=1;
cout<<"Enter the number:\n";
cin>>n;
do { f=f*n; n--; }
while(n>0);
cout<<"The factorial value is:";
cout <<j;
```

Problem understanding and Design (3 marks)	
Code Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

RESULT:

Thus the program is executed and output is verified for implement Simple Code Optimization techniques.

VIVA QUESTIONS
UNIT-I

1. What is a Complier?

A Complier is a program that reads a program written in one language-the source language-and translates it in to an equivalent program in another language-the target language .As an important part of this translation process, the compiler reports to its user the presence of errors in the source program

2. State some software tools that manipulate source program?

i. Structure editors
ii. Pretty printers
iii. Static
iv. checkers
v. Interpreters.

3. What are the cousins of compiler?

The following are the cousins of

i. Preprocessors
ii.Assemblers
iii. Loaders
iv. Link editors.

4. What are the main two parts of compilation? What are they performing?

The two main parts are

- Analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program.
- Synthesis part constructs the desired target program from the intermediate representation

5. What is a Structure editor?

A structure editor takes as input a sequence of commands to build a source program .The structure editor not only performs the text creation and modification functions of an ordinary text editor but it also analyzes the program text putting an appropriate hierarchical structure on the source program.

6. What are a Pretty Printer and Static Checker?

- A Pretty printer analyses a program and prints it in such a way that the structure of the program becomes clearly visible.
- A static checker reads a program, analyses it and attempts to discover potential bugs without running the program.

7. How many phases does analysis consists?

Analysis consists of three phases

i .Linear analysis
ii. Hierarchical analysis
iii. Semantic analysis

8. What happens in linear analysis?

This is the phase in which the stream of characters making up the source program is read from left to right and grouped in to tokens that are sequences of characters having collective

9. What happens in Hierarchical analysis?

This is the phase in which characters or tokens are grouped hierarchically in to nested collections with collective meaning.

10. What happens in Semantic analysis?

This is the phase in which certain checks are performed to ensure that the components of a program fit together meaningfully.

11. What is Lexical Analysis?

The first phase of compiler is Lexical Analysis. This is also known as linear analysis in which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters having a collective meaning.

12. What is a lexeme? Define a regular set.

- A Lexeme is a sequence of characters in the source program that is matched by the pattern for a token.
- A language denoted by a regular expression is said to be a regular set

13. What is a sentinel? What is its usage?

A Sentinel is a special character that cannot be part of the source program. Normally we use 'eof' as the sentinel. This is used for speeding-up the lexical analyzer.

14. What is a regular expression? State the rules, which define regular expression?

Regular expression is a method to describe regular language

Rules:

- ε-is a regular expression that denotes {ε} that is the set containing the empty string
- If a is a symbol in Σ,then a is a regular expression that denotes {a}
- Suppose r and s are regular expressions denoting the languages L(r) and L(s)
- Then,
    - (r)/(s) is a regular expression denoting L(r) U L(s).
    - (r)(s) is a regular expression denoting L(r)L(s)
    - (r)* is a regular expression denoting L(r)*.
    - (r) is a regular expression denoting L(r).

15. What are the Error-recovery actions in a lexical analyzer?

- Deleting an extraneous character
- Inserting a missing character
- Replacing an incorrect character by a correct character
- Transposing two adjacent characters

16. Construct Regular expression for the language

L= {w ε{a,b}/w ends in abb}

Ans: {a/b}*abb.

17. What is recognizer?

Recognizers are machines. These are the machines which accept the strings belonging to certain language. If the valid strings of such language are accepted by the machine then it is said that the corresponding language is accepted by that machine, otherwise it is rejected.

18. Differentiate compiler and interpreter.
Compiler produces a target program whereas an interpreter performs the operations implied by the source program.

19. Write short notes on buffer pair.
Concerns with efficiency issues Used with a lookahead on the input It is a specialized buffering technique used to reduce the overhead required to process an input character. Buffer is divided into two N-character halves. Use two pointers. Used at times when the lexical analyzer needs to look ahead several characters beyond the lexeme for a pattern before a match is announced.

20. Differentiate tokens, patterns, lexeme.

Tokens- Sequence of characters that have a collective meaning.
Patterns- There is a set of strings in the input for which the same token is produced as
output. This set of strings is described by a rule called a pattern associated with the token
Lexeme- A sequence of characters in the source program that is matched by the pattern for a token

UNIT-II

1. Define parser.
Hierarchical analysis is one in which the tokens are grouped hierarchically into nested collections with collective meaning. Also termed as Parsing.

2. Mention the basic issues in parsing.
There are two important issues in parsing.

- Specification of syntax
- Representation of input after parsing.

3. Why lexical and syntax analyzers are separated out?
Reasons for separating the analysis phase into lexical and syntax analyzers:

- Simpler design.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

4. Define a context free grammar.
A context free grammar G is a collection of the following

V is a set of non terminals
T is a set of terminals
S is a start symbol
P is a set of production rules
G can be represented as G = (V,T,S,P)

Production rules are given in the following form

Non terminal → (V U T)*

5. Briefly explain the concept of derivation.

Derivation from S means generation of string w from S. For constructing derivation two things are important.

- Choice of non terminal from several others.
- Choice of rule from production rules for corresponding non terminal.

Instead of choosing the arbitrary non terminal one can choose

- either leftmost derivation – leftmost non terminal in a sentinel form
- or rightmost derivation – rightmost non terminal in a sentinel form

6. Define ambiguous grammar.

A grammar G is said to be ambiguous if it generates more than one parse tree for some sentence of language L(G).i.e. both leftmost and rightmost derivations are same for the given sentence.

7. What is a operator precedence parser?

A grammar is said to be operator precedence if it possess the following properties:

- No production on the right side is ε.
- There should not be any production rule possessing two adjacent non terminals at the right hand side.

8. List the properties of LR parser.

- LR parsers can be constructed to recognize most of the programming languages for which the context free grammar can be written.
- The class of grammar that can be parsed by LR parser is a superset of class of grammars that can be parsed using predictive parsers.
- LR parsers work using non backtracking shift reduce technique yet it is efficient one.

9. Mention the types of LR parser.

- SLR parser- Simple LR parser
- LALR parser- LookAhead LR parser
- CLR - Canonical LR parser

10. What are the problems with top down parsing?

The following are the problems associated with top down parsing:

- Backtracking
- Left recursion
- Left factoring
- Ambiguity

UNIT-III

1. **What Are The Benefits Of Intermediate Code Generation?**

A Compiler for different machines can be created by attaching different back end to the existing front ends of each machine.

A Compiler for different source languages can be created by proving different front ends for corresponding source languages t existing back end.

A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.

2. **What Are The Various Types Of Intermediate Code Representation?**

There are mainly three types of intermediate code representations.

- Syntax tree
- Postfix
- Three address code

3.What is SDD?

SDD stands for syntax directed definition. It specifies attribute values by associating semantic rules with productions.

4. What is the difference between SDD and SDT?

SDT embeds program fragments, also called semantic actions, within production bodies. The position of the action defines the order in which the action is executed.

5,What are the advantages of SDT in compiler design?

SDT provides an easy way to specify transitions and grammar rules, and it makes it easy to translate programming languages. It also maintains the compiler and is easy to modify.

6. What is type checking in a compiler?

Type checking is the process of verifying that operations in a program are being performed on compatible data types. It is a part of the semantic analysis phase of the compiler, where the compiler checks whether the types of variables, expressions, and functions match the expected types as per the language's rules. Type checking helps in ensuring the program is logically correct and that no operations are performed on incompatible types (e.g., adding an integer to a string).

UNIT - IV

1. List the different storage allocation strategies.

The strategies are:

- Static allocation
- Stack allocation
- Heap allocation

2. What are the contents of activation record?

The activation record is a block of memory used for managing the information needed by a single execution of a procedure. Various fields f activation record are:

- Temporary variables
- Local variables
- Saved machine registers
- Control link
- Access link
- Actual parameters
- Return values

3. What is dynamic scoping?

In dynamic scoping a use of non-local variable refers to the non-local data declared in most recently called and still active procedure. Therefore, each time new findings are set up for local names called procedure. In dynamic scoping symbol tables can be required at run time.

4. Define symbol table.
Symbol table is a data structure used by the compiler to keep track of semantics of the variables. It stores information about scope and binding information about names.

5. What is code motion?
Code motion is an optimization technique in which amount of code in a loop is decreased. This transformation is applicable to the expression that yields the same result independent of the number of times the loop is executed. Such an expression is placed before the loop.

6. What are the properties of optimizing compiler?
The source code should be such that it should produce minimum amount of target code.There should not be any unreachable code. Dead code should be completely removed from source language.
The optimizing compilers should apply following code improving transformations on source language.

- common subexpression elimination
- dead code elimination
- code movement
- strength reduction

7. What are the various ways to pass a parameter in a function?

- Call by value
- Call by reference
- Copy-restore
- Call by name

8. Suggest a suitable approach for computing hash function.
Using hash function we should obtain exact locations of name in symbol table. The hash function should result in uniform distribution of names in symbol table. The hash function should be such that there will be minimum number of collisions. Collision is such a situation where hash function results in same location for storing the names.

9. Define bottom up parsing?
It attempts to construct a parse tree for an input string is beginning at leaves and working up towards the root (i.e.) reducing a string „w‟ to the start symbol of a grammar. At each reduction step, a particular substring matching the right side of a production is replaced by the symbol on the left of that production. It is a rightmost derivation and it‟ s also known as shifts reduce parsing.

10. What are the functions used to create the nodes of syntax trees?

- mknode (op, left, right)
- mkleaf (id,entry)
- mkleaf (num, val)

UNIT-V

1. Define code generations.
It is the final phase in compiler model and it takes as an input an intermediate representation of the source program and output produces as equivalent target programs. Then intermediate instructions are each translated into a sequence of machine instructions that perform the same task.

2. What are the issues in the design of code generator?

- Input to the generator
- Target programs
- Memory management
- Instruction selection
- Register allocation
- Choice of evaluation order
- Approaches to code generation.

3. Give the variety of forms in target program.

- Absolute machine language.
- Relocatable machine language.
- Assembly language.

4. Give the factors of instruction selections.

- Uniformity and completeness of the instruction sets
- Instruction speed and machine idioms
- Size of the instruction sets.

5. What are the sub problems in register allocation strategies?

During register allocation, we select the set of variables that will reside in register at a point in the program.

During a subsequent register assignment phase, we pick the specific register that a variable reside in.

6. Give the standard storage allocation strategies.

- Static allocation
- Stack allocation.

7. Define static allocations and stack allocations

Static allocation is defined as lays out for all data objects at compile time. Names are bound to storage as a program is compiled, so there is no need for a Run time support package.

Stack allocation is defined as process in which manages the run time as a Stack. It is based on the idea of a control stack; storage is organized as a stack, And activation records are pushed and popped as activations begin and end.

8. Write the addressing mode and associated costs in the target machine.

MODE FORM ADDRESS ADDED COST

- Absolute M M 1
- Register R R 0
- Indexed c(R) c+contents(R) 1
- Indirect register *R contents(R) 0
- Indirect indexed *c(R) contents(c+contents(R)) 1

9. Define basic block and flow graph.

A basic block is a sequence of consecutive statements in which flow of Control enters at the beginning and leaves at the end without halt or possibility Of branching except at the end.

A flow graph is defined as the adding of flow of control information to the Set of

basic blocks making up a program by constructing a directed graph.

10. Write the step to partition a sequence of 3 address statements into basic blocks.

- First determine the set of leaders, the first statement of basic blocks.
  The rules we can use are the following.
  - The first statement is a leader.
  - Any statement that is the target of a conditional or unconditional goto is a leader.
  - Any statement that immediately follows a goto or conditional goto statement is a leader.

- For each leader, its basic blocks consists of the leader and all statements Up to but not including the next leader or the end of the program.

****